

CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER

Escola Politécnica

Curso de Análise e Desenvolvimento de Sistemas

Raynan Vieira Tavares

RU 4825877

**PROJETO MULTIDISCIPLINAR — TRILHA BACK-END
REDE RAÍZES DO NORDESTE**

Relatório da atividade prática

Polo de apoio: EAD / Univirtus

Disciplina: Projeto Multidisciplinar

Professora: Profa. Me. Luciane Yanase Kanashiro

2026

SUMÁRIO

1 Introdução

2 Análise e requisitos

3 Modelagem e arquitetura

4 API e endpoints

5 LGPD, privacidade e segurança

6 Entrega técnica

7 Plano de testes

8 Conclusão

Referências

1 INTRODUÇÃO

Este trabalho apresenta a API REST desenvolvida para a rede de lanchonetes **Raízes do Nordeste**, no contexto da disciplina de Projeto Multidisciplinar (trilha Back-end). O estudo de caso descreve uma franquia em expansão, com matriz e unidades, cardápio por loja e atendimento em vários canais: aplicativo, totem, balcão, retirada (pick-up) e web.

O recorte executável segue o Fluxo A do roteiro da trilha: criar pedido, solicitar pagamento por serviço externo simulado (mock) e atualizar o status até a entrega. O canal de origem é tratado como dado de domínio (`canalPedido`). A solução foi organizada em camadas, persiste em PostgreSQL e documenta o contrato em Swagger/OpenAPI.

Usuários principais: cliente (App/Web/Totem), atendente, cozinha, gerente e administrador. O gateway de pagamento aparece como ator externo, sem processamento financeiro real nesta API.

2 ANÁLISE E REQUISITOS

2.1 CONTEXTO

A rede nasceu em Recife e opera unidades com estoque e equipe próprios. O cliente deve ver o cardápio da unidade, pedir, pagar por meio de um serviço externo e acompanhar o andamento. A matriz precisa de rastreamento por canal para consolidar a operação. O pagamento não é feito no sistema central: a API apenas solicita, registra o retorno e atualiza o pedido.

2.2 REQUISITOS FUNCIONAIS

A tabela abaixo traduz o estudo de caso e o roteiro da trilha. O recorte executável é o Fluxo A (seção 7.2 do roteiro). Os demais itens do levantamento constam como requisito do caso, para manter a análise alinhada ao enunciado.

ID	Requisito	Neste trabalho
RF01	Cadastro e autenticação de usuários com perfis (roles)	Implementado (JWT)
RF02	Consulta de cardápio (produtos) por unidade	Implementado
RF03	Criação de pedido com itens, valores e status	Implementado
RF04	Atualização de status (cozinha, pronto, entregue, cancelado)	Implementado
RF05	Campo obrigatório canalPedido (APP, TOTEM, BALCAO, PICKUP, WEB) e filtro na listagem	Implementado
RF06	Pagamento via serviço externo simulado, com registro do retorno	Implementado (mock)
RF07	Gestão de unidades da rede	Consulta implementada; cadastro apenas no seed
RF08	Controle de estoque por unidade (entrada/saída e restrição de venda)	Levantado no estudo de caso; fora do Fluxo A
RF09	Programa de fidelização (pontos, resgate, consentimento)	Levantado no estudo de caso; fora do Fluxo A
RF10	Promoções e campanhas	Levantado no estudo de caso; fora do Fluxo A

2.3 REQUISITOS NÃO FUNCIONAIS

ID	Requisito	Neste trabalho
----	-----------	----------------

RNF01	Autenticação por token e autorização por perfil	Implementado
RNF02	Senha armazenada com hash (bcrypt)	Implementado
RNF03	Padrão único de erro JSON	Implementado
RNF04	Documentação OpenAPI/Swagger	Implementado em /api
RNF05	Logs de ações sensíveis do fluxo de pedido	Implementado (AuditLog)
RNF06	Tolerância a falha no mock de pagamento	Implementado (simularFalha)
RNF07	Ambiente reproduzível para correção	Implementado (Compose + README)

3 MODELAGEM E ARQUITETURA

3.1 DIAGRAMA DE CASOS DE USO

Atores: Cliente (App/Web/Totem), Atendente, Cozinha, Gerente/Administrador e o gateway de pagamento (mock). Os casos destacados (realizar pedido e solicitar pagamento) detalham-se a seguir. Arquivo Draw.io: docs/diagramas/casos-de-uso.drawio.

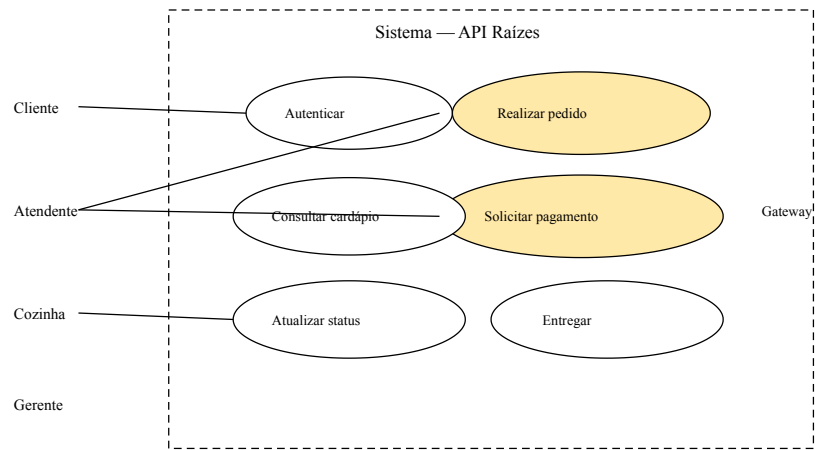


Figura 1 — Casos de uso (visão resumida)

UC — Realizar pedido e solicitar pagamento

Ator principal: Cliente (também Atendente, em nome do cliente).

Pré-condições: usuário autenticado; unidade e produtos existentes e ativos; `canalPedido` informado com valor do ENUM.

Fluxo principal:

1. O ator envia POST /pedidos com canal, unidade e itens.
2. AAPI valida unidade, produtos e quantidade mínima.
3. O pedido é persistido com status AGUARDANDO_PAGAMENTO e total calculado.
4. O ator envia POST /pedidos/{id}/pagamentos com o resultado do mock.
5. AAPI grava o pagamento e atualiza o pedido para PAGO ou PAGAMENTO_NEGADO.
6. A cozinha move PAGO → EM_PREPARO → PRONTO; o atendente move PRONTO → ENTREGUE.

Pós-condições: pedido e pagamento persistidos; auditoria registrada na criação, no pagamento e na mudança de status.

Exceções: 401 sem token; 403 perfil indevido; 400/422 canal ausente ou inválido; 404 unidade/produto inexistente; 409 transição de status ilegal; 502 quando o mock simula falha de infraestrutura (pedido permanece no estado anterior à confirmação).

3.2 DER

Entidades alinhadas ao Prisma: Usuario, Unidade, Produto, Pedido, PedidoItem, Pagamento e AuditLog. Cardinalidades: uma unidade possui vários produtos e pedidos; um pedido possui vários itens e vários registros de pagamento; o pagamento fica desacoplado do pedido. Arquivo: docs/diagramas/der.drawio.

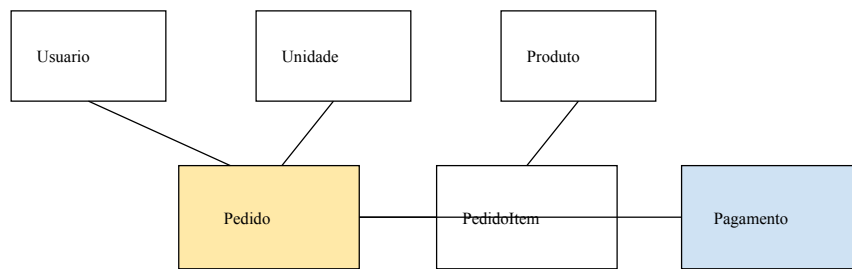


Figura 2 — DER (entidades principais)

3.3 ARQUITETURA EM CAMADAS

Domain concentra a máquina de estados do pedido. Application orquestra login, criação de pedido e pagamento mock. A infraestrutura isola Prisma, JWT e auditoria. API expõe controllers, DTOs, guards e Swagger. Arquivo:

`docs/diagramas/arquitetura.drawio.`

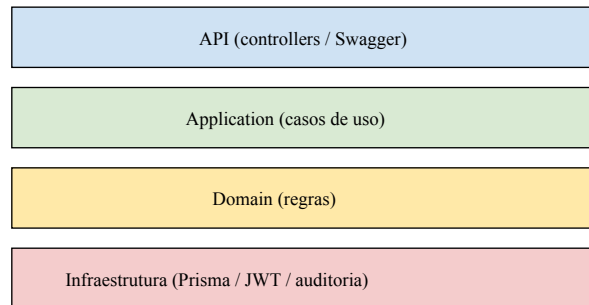


Figura 3 — Camadas

3.4 DIAGRAMA DE CLASSES E SEQUÊNCIA

O diagrama de classes de domínio está em `docs/diagramas/classes.drawio`. Pedido agrega itens; Pagamento associa-se a um Pedido sem processar adquirente real.

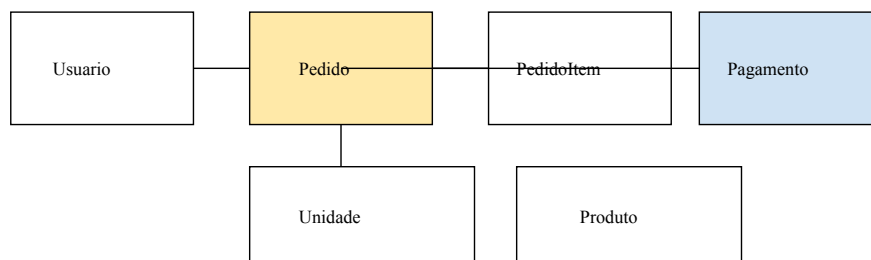


Figura 4 — Classes de domínio

A sequência do fluxo crítico está em `docs/diagramas/sequencia-fluxo-a.drawio`

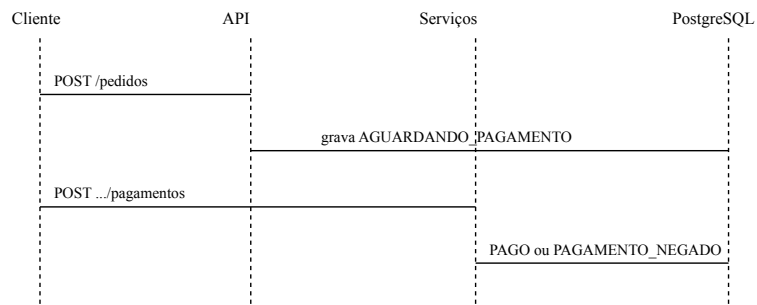


Figura 5 — Sequência do Fluxo A

4 API E ENDPOINTS

Padrão de erro:

```

{
  "error": "NOME_DO_ERRO",
  "message": "Mensagem legível",
  "statusCode": 400,
  "details": [{ "field": "canalPedido", "message": "..."}],
  "timestamp": "2026-08-16T12:00:00.000Z",
  "path": "/pedidos",
  "requestId": "uuid"
}
  
```

/auth — Login

Finalidade: autenticar e devolver JWT. Método: POST /auth/login. Auth: público.

```

Request: { "email": "cliente@raizes.local", "senha": "Senha@123" }
Response 201: { "accessToken": "...", "tokenType": "Bearer", "user": { "id", "nome", "email", "role" } }
Status: 201, 400, 401
  
```

/auth — Cadastro

POST /auth/register. Cria usuário com papel CLIENTE. 201 ou 409 se o e-mail já existe.

/unidades e /produtos

GET /unidades — JWT. GET /produtos?unidadeId= — JWT. GET /produtos/{id} — JWT.

/pedidos — Criar

POST /pedidos. JWT. Papéis: CLIENTE, ATENDENTE, GERENTE, ADMIN.

```

{
  "canalPedido": "TOTEM",
  "unidadeId": "uuid",
  "itens": [{ "produtoId": "uuid", "quantidade": 2 }]
}
  
```

201 com status AGUARDANDO_PAGAMENTO. 400/422 sem canalPedido. 404 produto/unidade inválidos.

/pedidos — Listar e detalhar

GET /pedidos?canalPedido=TOTEM&status=PAGO&unidadeId=&page=1&limit=10. GET /pedidos/{id}. CLIENTE só vê os próprios pedidos.

/pedidos/{id}/status

PATCH. Papéis: ATENDENTE, COZINHA, GERENTE, ADMIN. Corpo: { "status": "EM_PREPARO" }. 409 se a transição for ilegal. Cozinha só EM_PREPARO e PRONTO. Cancelamento restrito a GERENTE e ADMIN.

/pedidos/{id}/pagamentos

POST. Corpo: { "aprovado": true } ou false; opcional simularFalha. Forma MOCK. Atualiza o pedido para PAGO ou PAGAMENTO_NEGADO.

5 LGPD, PRIVACIDADE E SEGURANÇA

Dados pessoais tratados: nome, e-mail e identificador do usuário vinculados ao pedido. pedido e rastreo operacional. Base: execução do serviço solicitado pelo titular (pedido) e consentimento implícito no cadastro acadêmico de demonstração.

Minimização: a API não devolve senha nem hash. Papel CLIENTE não acessa pedido de terceiros. Senha só em bcrypt. Autorização por role nos endpoints do fluxo. Ações de criar pedido, pagar e mudar status geram AuditLog (usuarioId, acao, entidade, detalhes). Seed usa e-mails fictícios (@raizes.local). Não há retenção em provedor externo; o banco é local.

6 ENTREGA TÉCNICA

Repositório público: <https://github.com/raynantavares/raizesd0n0rdeste00.git>

Swagger local: <http://localhost:3000/api>

Coleção: postman/Raizes-Nordeste-Fluxo-A.postman_collection.json

Ambiente: postman/Raizes-Nordeste-local.postman_environment.json

Pré-requisitos: Node.js 20+, Docker (apenas PostgreSQL), npm. Passos: copiar .env.example para .env; npm install; npm run docker:up; npx prisma migrate deploy; npm run db:seed; npm run start:dev.

Usuários do seed (senha Senha@123): cliente@raizes.local, atendente@raizes.local, cozinha@raizes.local, gerente@raizes.local, admin@raizes.local.

Stack: Node.js, NestJS, PostgreSQL 16, Prisma 5, JWT, Swagger, Draw.io. Migrations em prisma/migrations. O DER corresponde às tabelas geradas.

7 PLANO DE TESTES

ID	Cenário	Endpoint	Pré-condição	Esperado	Evidência
T01	Login válido	POST /auth/login	seed	201 + accessToken	Auth / T01 Login válido (CLIENTE)
T02	Listar unidades	GET /unidades	token cliente	200	Catalogo / T02
T03	Listar produtos da unidade	GET /produtos	unidadeId	200	Catalogo / T03

T04	Criar pedido com canal TOTEM	POST /pedidos	itens válidos	201 AGUARDANDO_PAGAMENTO	Pedidos / T04
T05	Filtrar por canal	GET /pedidos? canalPedido=TOTEM	pedido criado	200	Pedidos / T05
T06	Pagamento mock aprovado	POST .../pagamentos	pedido aberto	201 pedido PAGO	Pagamentos / T06
T07	Cozinha EM_PREPARO e PRONTO	PATCH .../status	pedido PAGO	200	Status / T07 e T07b
T08	Atendente marca ENTREGUE	PATCH .../status	pedido PRONTO	200 ENTREGUE	Status / T08
T09	Sem token	GET /pedidos	—	401 erro padrão	Erros / T09
T10	Cliente altera status	PATCH .../status	token CLIENTE	403 SEM_PERMISSAO	Erros / T10
T11	canalPedido ausente	POST /pedidos	token	400 details.field canalPedido	Erros / T11
T12	Produto inexistente	POST /pedidos	UUID inválido de produto	404	Erros / T12
T13	Transição ilegal de status	PATCH .../status	pedido AGUARDANDO_PAGAMENTO → ENTREGUE	409 TRANSICAO_INVALIDA	Erros / T13
T14	Pagamento mock recusado	POST .../pagamentos	aprovado false	201 PAGAMENTO_NEGADO	Pagamentos / T14

T01 a T08 são caminhos esperados (positivos). T09 a T14 cobrem autenticação, autorização, validação, regra de negócio (409) e pagamento recusado. Auditoria: cada T04, T06 e T07 grava linha em AuditLog; a API não expõe GET de log. Conferência opcional no banco: tabela AuditLog após o runner.

8 CONCLUSÃO

Foi entregue uma API executável do Fluxo A, com persistência, JWT e papéis, canalPedido, pagamento mock (aprovado, recusado e falha), Swagger, coleção Postman e diagrama alinhado ao schema. Casos de uso, DER e classes descrevem o mesmo fluxo dos endpoints POST /pedidos e POST /pedidos/{id}/pagamentos. O erro JSON único aparece nos testes negativos. Segurança limitada ao necessário do recorte: hash de senha, token, papéis e auditoria das ações do pedido.

REFERÊNCIAS

FIELDING, Roy. Architectural Styles and the Design of Network-based Software Architectures. Tese (Doutorado) — University of California, Irvine, 2000.

Raynan Vieira Tavares

RU 4825877

Centro Universitário Internacional UNINTER

Projeto Multidisciplinar — trilha Back-end

Rede Raízes do Nordeste

Evidências complementares

Repositório: <https://github.com/raynantavares/raizesd0n0rdeste00.git>

EVIDÊNCIAS DE DESENVOLVIMENTO E TESTES

Este anexo reúne prints e logs gerados na máquina de desenvolvimento para comprovar autoria, histórico de versões, documentação Swagger, execução dos testes automatizados da coleção Postman (Newman) e registro de auditoria no banco PostgreSQL após o Fluxo A.

1. Repositório GitHub público

URL: <https://github.com/raynantavares/raizesd0n0rdeste00.git>

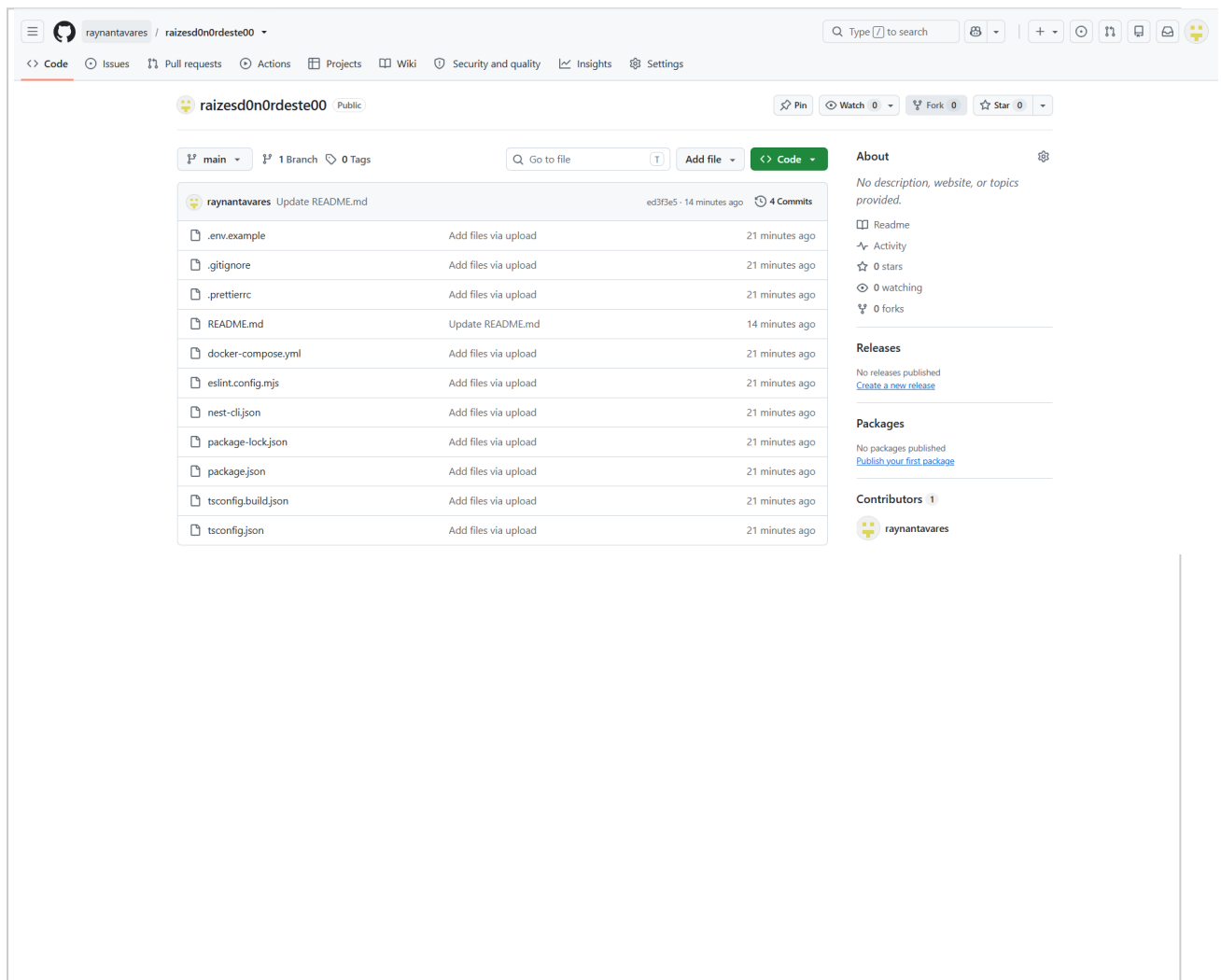


Figura 1 — Repositório no GitHub

2. Histórico de commits

Oito commits em português, com evolução incremental do projeto (setup → banco → API → README → Postman → diagramas → relatório).

```
fe3fad2 docs: inclui figuras de classes e sequência no relatório
1fff2e1 docs: adiciona o relatório acadêmico do projeto
73d3ca0 docs: adiciona diagramas de casos de uso, DER, classes e camadas
7c68b77 test: adiciona coleção Postman do fluxo principal
1b61dbb docs: descreve execução local no README
2758f46 feat(api): implementa autenticação JWT, pedidos e pagamento mock
d7ee3ff feat(db): adiciona schema Prisma, migration e seed
e9b0824 chore: inicializa o projeto NestJS e o ambiente local
```

3. Swagger / OpenAPI

Documentação interativa em <http://localhost:3000/api> (API local após `npm run start:dev`).

 **Swagger**
supported by SMARTBEAR

Raízes do Nordeste API 1.0 OAS 3.0

API Back-end — Projeto Multidisciplinar Uninter (,) Fluxo A: Pedido → Pagamento mock → Status.

Authorize 

auth

POST

/auth/register

Cadastrar usuário

▼

POST

/auth/login

Autenticar e obter JWT

▼

pedidos

POST

/pedidos

Criar pedido (canalPedido obrigatório)

 ▼

GET

/pedidos

Listar pedidos (filtro por canalPedido/status)

 ▼

GET

/pedidos/{id}

Detalhar pedido

 ▼

PATCH

/pedidos/{id}/status

Atualizar status do pedido

 ▼

POST

/pedidos/{id}/pagamentos

Solicitar pagamento mock do pedido

 ▼

catalogo

GET

/unidades

Listar unidades ativas

 ▼

GET

/produtos

Listar produtos (opcionalmente por unidade)

 ▼

GET

/produtos/{id}

Detalhar produto

 ▼

Schemas

RegisterDto >

LoginDto >

PedidoItemDto >

CriarPedidoDto >

AtualizarStatusDto >

SolicitarPagamentoDto >

Figura 3 — Swagger com endpoints auth, unidades, produtos e pedidos

4. Execução da coleção Postman (Newman)

Comando: `npx newman run postman/Raizes-Nordeste-Fluxo-A.postman_collection.json -e postman/Raizes-Nordeste-local.postman_environment.json`

iterations	1	0
requests	19	0
test-scripts	19	0
assertions	34	0
total run duration: 653ms		

Cenários: T01-T08 (fluxo positivo Fluxo A), T09-T14 (401, 403, 400, 404, 409, pagamento recusado).

5. Build e health check

`npm run build` → sucesso (nest build)
`GET /api` → HTTP 200
`GET /unidades (sem token)` → HTTP 401

6. Auditoria no banco (AuditLog)

acao	entidade	createdAt
PEDIDO_CRIADO	Pedido	2026-08-22 20:51:14.253
PEDIDO_STATUS_ATUALIZADO	Pedido	2026-08-22 20:51:14.171
PEDIDO_STATUS_ATUALIZADO	Pedido	2026-08-22 20:51:14.15
PEDIDO_STATUS_ATUALIZADO	Pedido	2026-08-22 20:51:14.129
PAGAMENTO MOCK	Pagamento	2026-08-22 20:51:14.108
PEDIDO_CRIADO	Pedido	2026-08-22 20:51:14.085
PAGAMENTO MOCK	Pagamento	2026-08-22 20:51:14.062
PEDIDO_CRIADO	Pedido	2026-08-22 20:51:14.012

7. Arquivos brutos nesta pasta

- `01-historico-commits.txt` — log completo do Git
- `03-newman-log.txt` — saída integral do Newman
- `03-newman-resultado.json` — resultado estruturado
- `06-auditoria-banco.txt` — consulta SQL

- `07-build-e-healthcheck.txt` — build e HTTP checks